项目4 企业网站的产品展示页设计

企业网站的产品展示页设计在网页设计领域中属于多媒体播放类页面的设计项目，其设计目的是将目标网页打造成一个类似于哔哩哔哩的多媒体播放界面，以便利用视频、动画等多媒体资料更直观地向公众展示网站所属企业的主打产品与经营近况。在此类项目中，网页设计师们除了需要利用本书在上一章中介绍的方法来对网页中的图文信息进行排版之外，通常还需要充分利用HTML中的多媒体类标记以及Bootstrap框架提供的相关组件来构建基于浏览器的多媒体播放界面。由于多媒体形式的宣传具有更为直观的传播能力，这将有助于为企业网站吸引到更多潜在的客户或合作伙伴，所以此类项目也被认为是网页设计师们在实际工作中最常接到的任务之一。

【学习目标】

在本章，笔者会继续以凌雪冰熊网站中“产品展示”页的设计为例来演示如何为企业网站设计出类似于哔哩哔哩的多媒体播放界面。该项目的设计目标是为凌雪冰熊网站提供视频、动画等多媒体资料的播放页面，以便更直观地展示这家连锁店最近的主打产品与新加盟的门店，从而吸引到更多潜在的合作伙伴。同样的，该网页在外观设计上也必须要延续该网站首页建立起来的布局风格与配色方案，并同样在导航栏中提供跳转到网站首页、新闻活动、申请加盟等页面的链接。通过本章项目的实践，读者将会初步了解设计一个多媒体播放类页面所要执行的基本步骤，以及执行这些步骤所需的基本技术与相关工具。总而言之，在阅读完本章之后，我们希望读者能够：

- 掌握如何利用Bootstrap框架提供的组件来完成针对多媒播放体类页面的布局任务；
- 掌握HTML 5中提供的多媒体类标记，并将这些标记具体运用到网页设计项目中；

【学习场景描述】

凌雪冰熊连锁店的网页设计团队如今已经完成了其官方网站的首页设计，并基于该设计进一步创建了该网站的网页模板。现在，他们希望你能基于该模板继续为该网站设计用于播放多媒体资料的页面，目的是更好地宣传凌雪冰熊这家连锁店最近的主打产品，以及在全国各地新加盟的门店，从而进一步展现企业品牌的竞争力。在这个网页设计项目中，你的主要任务是为网站的“产品展示”页设计一个类似于哔哩哔哩的多媒体播放页面，以便凌雪冰熊官方团队可以利用该页面发布多媒体形式的宣传资料。当然了，你同样需要确保该页面采用与首页一致的布局风格与配色方案。

【任务书】

- **项目名**：凌雪冰熊网站的产品展示页设计
- **委托方**：凌雪冰熊股份有限公司互联网部门
- **项目资料**：
    - **代码资料**：凌雪冰熊官方网站现有的设计源码；
    - **多媒体资料**：凌雪冰熊官方提供的音频、视频等多媒体资料；
- **项目要求**：为凌雪冰熊连锁饮料店的官方网站设计首页，该网页的设计应符合以下要求。
    - 该网页需要为网站的客户提供用户体验良好的、多媒体播放界面；
    - 该网页在外观样式上需要采用与网站首页一致的布局风格与配色方案；
- 时间要求：在3个工作日内完成；

【任务拆解】

本章项目的实施过程可以划分为以下三个小任务来进行：

- 基于凌雪冰熊官方网站提供的网页设计模版来创建该网站的产品展示页；
- 利用Bootstrap框架创建多媒体播放界面，包括媒体列表与媒体播放区域；
- 利用HTML标记将凌雪冰熊官方提供的多媒体资料填充到刚刚创建的界面中；

【工作准备】

在经过了上一章的项目实践之后，读者想必已经对如何安排网页中的图文类信息有了一个基本的了解。在本章接下来要实践的这一类项目中，设计师们将要在页面中设置的是更为复杂的多媒体元素及其外观样式，这需要读者学习更多的HTML标记与相关的Bootstrap组件。下面，笔者将根据本章项目的任务需求来继续为大家介绍相关的知识点和工具，同样的，如果读者自认为已经掌握了这部份知识，也可以选择跳过本节内容，直接进入本章项目的【工作实施与交付】环节。

知识点2：HTML 5提供的多媒体类标记

在网页设计工作中，除了最基本的图文类元素之外，设计师们通常还会根据任务的需求在网页中嵌入音频、视频、矢量图、画布等特定媒体类型的页面元素，以便在网页中呈现更多媒体形态的内容。这些元素也都有对应的HTML标记。下面，笔者将会分别来介绍一下这些HTML标记，以便读者可以根据项目的任务需求来设置网页中要呈现的内容。

音频与视频

在HTML 5中，设计师们可以使用 `<video>`、`<audio>`这两个标记来实现在网页中嵌入视频/音频元素，如今人们所熟悉的哔哩哔哩、喜马拉雅等视频/音频网站，就是基于这两个标记来实现的，下面来分别介绍一下它们的使用方法：

- `<audio>`标记：该标记用于在网页文档中嵌入一个音频播放器元素，设计师们可以利用其`<source>`子标记的`src`属性来指定要播放的音频文件，例如像这样：

  ```
  <!DOCTYPE html>
  <html>
      <head>
          <title>嵌入音频播放器</title>
      </head>
      <body>
          <audio width="400" height="300" controls>
              <source src="horse.mp3" type="audio/mpeg">
              <p>你的浏览器不支持HTML 5的音频标签！ </p>
          </audio>
      </body>
  </html>
  ```

  上述示例被保存在本书源码包中的`Examples/00_demo/embedCase`目录下的`index.htm`文件中，读者可以使用网页浏览器打开该文件，就可以看到如图4-1所示的效果：

图4-1：嵌入音频播放器

接下来，让我们根据上述示例来简单介绍一下在网页中嵌入音频播放器元素的基本步骤，具体如下：

- 首先要做的是使用 `<audio>` 标记来在网页中创建一个音频播放器元素，并设置该元素的位置、高度、宽度以及其他与播放器相关的属性。在此过程中，设计师们主要会使用到该标记的以下属性：
  - `width` 和 `height`属性：这两个属性主要用于设置音频播放器元素在网页中所要显示的高度和宽度。当然在这里，笔者更倾向于建议大家用CSS样式来设置这部分内容，而不是直接在HTML文档中使用这两个属性。
  - `autoplay`属性：该属性用于指定是否要在网页加载完毕之后自动开始播放音频。
  - `controls`属性：该属性用于指定是否要在网页中显示音频播放器的控制按钮。
  - `loop`属性：该属性用于指定是否要在网页中循环播放音频。
  - `preload`属性：该属性用于指定是否要在网页加载完毕之后预先加载音频文件。
  - `muted`属性：该属性用于指定是否要在网页中默认将音频静音。
- 在定义好音频播放器元素之后，接下来要做的是使用其 `<source>` 子标记来指定该播放器元素具体要播放的媒体文件，该子标记的常用属性如下：
  - `src` 属性：该属性用于指定要媒体文件所在的位置，通常以URL的形式来表示。
  - `type`属性：该属性用于指定要播放的媒体文件的类型。到目前为止，在网页中可以播放的音频格式主要有以下几种：
    - `audio/mpeg`：该属性用于指定要播放的媒体文件为MPEG格式的音频文件。
    - `audio/ogg`：该属性用于指定要播放的媒体文件为Ogg格式的音频文件。
    - `audio/wav`：该属性用于指定要播放的媒体文件为WAV格式的音频文件。
    - `audio/webm`：该属性用于指定要播放的媒体文件为WebM格式的音频文件。
- 最后，由于考虑到目前市面上各种浏览器对HTML 5的支持程度并不一致，设计师们通常还会在`<audio>`标记内部设置一些文本类标记（例如 `<p>` ），用来指定当浏览器不支持HTML 5的音频标签时页面中要显示的提示信息。

- `<video>`标记：该标记用于在网页文档中嵌入一个视频播放器元素，设计师们可以利用其`<source>`子标记的`src`属性来指定要播放的视频文件，例如像这样：

```
<!DOCTYPE html>
<html>
    <head>
        <title>嵌入视频播放器</title>
    </head>
    <body>
```

```
        <video width="320" height="240" controls>
            <source src="movie.mp4" type="video/mp4">
            <p>你的浏览器不支持HTML 5的视频标签！ </p>
        </video>
    </body>
</html>
```

同样的，上述示例也被保存在本书源码包中的`Examples/00_demo/embedCase`目录下的`index.htm`文件中，读者可以使用网页浏览器打开该文件，就可以看到如图4-2所示的效果：

图4-2：嵌入视频播放器

接下来，让我们根据上面的示例来简单介绍一下在网页中嵌入视频播放器元素的基本步骤，具体如下：

- 首先要做的是使用`<video>`标记来在网页中创建一个视频播放器元素，并设置该元素的位置、高度、宽度以及其他与播放器相关的属性。在此过程中，设计师们主要会使用到该标记的以下属性：
  - `width` 和 `height`属性：这两个属性主要用于设置视频播放器元素在网页中所要显示的高度和宽度。当然在这里，笔者更倾向于建议大家用CSS样式来设置这部分内容，而不是直接在HTML文档中使用这两个属性。
  - `autoplay`属性：该属性用于指定是否要在网页加载完毕之后自动开始播放视频。
  - `controls`属性：该属性用于指定是否要在网页中显示视频播放器的控制按钮。
  - `loop`属性：该属性用于指定是否要在网页中循环播放视频。
  - `preload`属性：该属性用于指定是否要在网页加载完毕之后预先加载视频文件。
  - `muted`属性：该属性用于指定是否要在网页中默认将视频静音。
  - `poster`属性：该属性用于指定是否要在网页中默认显示视频的封面图片。
- 在定义好音频播放器元素之后，接下来要做的是使用其 `<source>` 子标记来指定该播放器元素具体要播放的媒体文件，该子标记的常用属性如下：
  - `src` 属性：该属性用于指定要媒体文件所在的位置，通常以URL的形式来表示。
  - `type`属性：该属性用于指定要播放的媒体文件的类型。到目前为止，在网页中可以播放的视频格式主要有以下几种：
    - `video/mp4`：该格式用于指定要播放的媒体文件为MP4格式。
    - `video/webm`：该格式用于指定要播放的媒体文件为WebM格式。
    - `video/ogg`：该格式用于指定要播放的媒体文件为Ogg格式。
- 最后，由于考虑到目前市面上各种浏览器对HTML 5的支持程度并不一致，设计师们通常还会在`<video>`标记内部设置一些文本类标记（例如 `<p>` ），用来指定当浏览器不支持HTML 5的音频标签时页面中要显示的提示信息。

矢量图与动画

- `<svg>`标记：该标记用于在网页文档中嵌入矢量图元素。在这里，矢量图是一种用数学公式来绘制的图形格式。由于人们在绘制矢量图时使用的是直线、矩形、圆形等数学几何图形而不是像素点，这使得它在被无限放大或缩小时完全不必担心图像显示会失真，所以矢量图这种图形格式通常要比我们之前使用的位图更适用于响应式网页的设计。

  在使用`<svg>`标记定义矢量图元素的过程中，设计师们需要用到一套基于XML实现的、名为SVG的计算机标记语言。例如，如果读者想在网页中嵌入一个绘制有红色圆形+黄色矩形的矢量图，就可以这样做：

```html
<!DOCTYPE html>
<html lang="zh-CN">
    <head>
        <meta charset="UTF-8">
        <title>嵌入矢量图</title>
    </head>
    <body>
        <svg width="400" height="300">
            <!-- 在这里放置用于绘制SVG图形的标记 -->
            <circle cx="50" cy="50" r="40"
                stroke="black" stroke-width="3" fill="red" />
            <rect x="100" y="100" width="200" height="100"
                stroke="black" stroke-width="3" fill="yellow" />
        </svg>
    </body>
</html>
```

  同样的，上述示例也会被保存在本书源码包中的`Examples/00_demo/embedCase`目录下的`index.htm`文件中，读者可以使用网页浏览器打开该文件，就可以看到如图4-3所示的效果。

  图4-3：嵌入矢量图

  接下来，让我们根据上面的示例简单介绍一下`<svg>`及其子标记的使用要旨：

  - `<svg>`标记具有开始标记 `<svg>` 和结束标记 `</svg>`，在这两个标记之间的内容将被渲染为SVG图形。设计师们可以使用该标记的`width`和`height`属性来指定图形的宽度和高度。这决定了SVG画布的尺寸，所有的图形元素将在这个画布上绘制。

  - SVG拥有一个独立的坐标系，该坐标系的起点（即`(0,0)`）通常位于`<svg>`标记所定义元素的左上角。设计师们可以在SVG中使用该坐标系来放置和定位图形元素。`<svg>`标记内的坐标系统是相对的，它们与`width`和`height`属性的值相关联。在 `<svg>` 标记内，设计师们可以使用一系列子标记来绘制不同的SVG图形元素，例如 `<circle>`、`<rect>`、`<line>`、`<path>` 等，这些子标记有各自的属性，可用于控制图形的外观和行为。
  - 设计师们可以使用CSS样式来控制SVG图形元素的颜色、填充、描边等外观属性。这些样式可以通过在网页中嵌入内联样式或者引用外部CSS文件来进行定义。
  - SVG图形中也可以包含交互性功能，例如添加鼠标事件处理程序，使用户能够与图形进行互动。另外，SVG支持动画，设计师们可以使用 `<animate>` 标记或 JavaScript 来为图形元素添加动画效果。
  - 设计师们可以将SVG图形嵌入到网页中，也可以通过外部文件引入 SVG 图形。这使得图形的重用和维护变得更加容易。

  总而言之，`<svg>`标记是一个可用于在网页中创建矢量图形和图表的强大工具，它提供了丰富的功能，包括绘制、样式、交互性和动画等。而且，SVG图形还可以在不失真的情况下缩放，适合多种不同的屏幕尺寸和分辨率。

- `<canvas>`标记：该标记用于在网页文档中嵌入可用于在运行时进行绘画的画布元素。在使用该标记创建该画布元素时，设计师们通常会先使用该标签的`width`和`height`属性来设置画布的宽度和高度，然后通过执行JavaScript脚本的方式来完成在该元素内部的绘画任务，例如像下面这样：

```html
<!DOCTYPE html>
<html>
    <head>
        <title>嵌入画布元素</title>
    </head>
    <body>
        <canvas id="canvas" width="320" height="240"></canvas>
        <script>
            const canvas = document.getElementById('canvas');
            const context = canvas.getContext('2d');
            context.fillStyle = '#FF0000';
            context.fillRect(0, 0, 150, 100);
        </script>
    </body>
</html>
```

  同样的，上述示例也会被保存在本书源码包中的`Examples/00_demo/embedCase`目录下的`index.htm`文件中，读者可以使用网页浏览器打开该文件，就可以看到如图4-4所示的效果：

嵌入画布元素：

```
<canvas id="canvas" width="320" height="240"></canvas>
<script>
  const canvas = document.getElementById('canvas');
  const context = canvas.getContext('2d');
  context.fillStyle = '#FF0000';
  context.fillRect(0, 0, 150, 100);
</script>
```

图4-4：嵌入画布元素

请注意，`<canvas>`标记与之前介绍的`<svg>`标记是不同的，`<svg>`标记所显示的是页面在被载入时由浏览器依据SVG语言的描述所绘制的数学几何图形，它通常是预先定义好的矢量图形。而`<canvas>`标记中的图形则是利用JavaScript脚本在其运行过程中依据指定参数来绘制的动态图形，因此它具有很大的可编程性，除了用于绘制静态图形之外，也可以用于制作动画和游戏。当然了，对于不熟悉JavaScript这门编程语言的网页设计师来说，在使用`<canvas>`标记之前可能需要学习一些与Canvas API相关的知识。另外，由于不同的网页浏览器对Canvas API的支持程度也有所不同，因此建议读者在使用时该标记时需要保持谨慎的态度，事前必须进行充分的兼容性测试。

知识点2：Bootstrap框架提供的相关组件

在之前的章节中，笔者已经介绍过如何使用轮播、卡片、折叠卡片等由Bootstrap框架提供的专用组件来安排网页中的图文信息类元素，而对于呈现形式更为复杂的多媒体类元素，设计师们需要用到一些结构更为复杂、功能也更为强大的专用组件。在本节中，笔者将继续为大家介绍这部分的Bootstrap组件，以便读者们可以更好地完成本章的项目任务。

- **选项卡组件**：如果读者想以选项卡的方式来对页面中所呈现的内容进行分门别类地组织，可以考虑使用Bootstrap框架中提供的选项卡组件来进行辅助设计，下面是该组件的一个简单示例：

```
<div id="tabdemo">
    <ul class="nav nav-tabs" role="tablist">
        <li class="nav-item" role="presentation">
            <button class="nav-link active" id="tab1-tab"
                data-bs-toggle="tab" data-bs-target="#tab1"
                aria-controls="tab1" aria-selected="true">
                选项卡 1
            </button>
        </li>
        <li class="nav-item" role="presentation">
            <button class="nav-link" id="tab2-tab"
                data-bs-toggle="tab" data-bs-target="#tab2"
                aria-controls="tab2" aria-selected="false">
                选项卡 2
            </button>
        </li>
        <li class="nav-item" role="presentation">
```

```
            <button class="nav-link" id="tab3-tab"
                data-bs-toggle="tab" data-bs-target="#tab3"
                aria-controls="tab3" aria-selected="false">
                选项卡 3
            </button>
        </li>
        <li class="nav-item" role="presentation">
            <button class="nav-link disabled" id="tab4-tab"
                data-bs-toggle="tab" data-bs-target="#tab4"
                aria-controls="tab4" aria-selected="false">
                禁用选项卡
            </button>
        </li>
    </ul>
    <div class="tab-content container p-3">
        <div class="tab-pane fade show active" id="tab1"
                role="tabpanel" aria-labelledby="tab1-tab">
            这是选项卡1的内容
        </div>
        <div class="tab-pane fade" id="tab2" role="tabpanel"
            aria-labelledby="tab2-tab">
            这是选项卡2的内容
        </div>
        <div class="tab-pane fade" id="tab3" role="tabpanel"
            aria-labelledby="tab3-tab">
            这是选项卡3的内容
        </div>
        <div class="tab-pane fade" id="tab4" role="tabpanel"
            aria-labelledby="tab4-tab">
            这是禁用选项卡的内容
        </div>
    </div>
</div>
```

接下来，让我们根据上面的示例来简单介绍一下与该组件相关的样式类及其使用方法，具体如下：

- `nav-tabs`：该样式类通常作用于设置了`nav`样式类的`<ul>`标记，效果是将该无序列表定义为选项卡组件的导航栏部分。另外，如果想让选项卡组件正常工作，我们还需为该`<ul>`标记设置`role`属性，并将该属性的值设置为`tablist`；

- `nav-item`：该样式类是和`nav`和`nav-tabs`这两个类的次级样式类，通常作用于设置了`nav-tabs`样式类的`<ul>`标记内部的各`<li>`标记，效果是将这些列表项设置为选项卡组件中导航栏部分的各个子项。同样，如果想让这些导航项正常工作，我们还需为这些`<li>`标记设置`role`属性，并将该属性的值设置为`presentation`；

- `nav-link`：该样式类用于设置选项卡组件中导航项的样式，通常作用于设置了`nav-item`样式类的`<li>`内部的`<a>`或`<button>`标记，效果是将该标记设置为链接样式；在设置这些导航项的样式时，读者需要注意以下事项：

    - 如果想让当前导航项处于默认被激活的状态，就需要在`nav-link`类后面再加上`active`样式类；
    - 如果想让当前导航项处于禁用状态，就需要在`nav-link`类后面再加上`disabled`样式类；

- `tab-content`：该样式类通常作用于选项卡组件中紧跟着导航栏部分后面的`<div>`标记，效果是将该标记定义的元素设置为充当组件中各选项卡元素的容器；
- `tab-pane`：该样式类通常作用于设置了`tab-content`样式类的`<div>`标记内第一级的各个`<div>`标记，效果是将这些标记设置为选项卡组件中的各个选项卡；在设置这些选项卡元素时，读者需要注意以下事项：
    - 每个`<div>`标记都应该有一个`id`属性，该属性的值应该与被设置了`nav-link`类的标记中`aria-controls`属性的值相同；
    - 如果想让组件中的各个选项卡都能正常发挥作用，我们就需要为这些`<div>`标记设置`role`属性，并将该属性的值设置为`tabpanel`；
    - 如果想赋予组件中的各选项卡元素在被切换时有淡入淡出的效果，我们就需要在`tab-pane`类后面再加上`fade`样式类；
    - 如果想让组件中的某个选项卡在页面载入时默认显示，我们就需要回到该选项卡所在的`<>`标记的`class`属性中，在`tab-pane`类后面再加上`show`样式类；
    - 如果想让组件中的某个选项卡默认处于激活状态，我们就需要回到该选项卡所在的`<>`标记的`class`属性中，在`tab-pane`类后面再加上`active`样式类；
- **分页导航组件**：如果读者想让页面中的内容分页显示，可以考虑使用Bootstrap框架中提供的分页组件来进行辅助设计，下面是该组件的一个简单示例：

```html
<nav id="paginationExample" aria-label="Page navigation example">
    <ul class="pagination">
        <li class="page-item disabled">
            <a class="page-link" href="#" tabindex="-1"
                aria-disabled="true">Previous</a>
        </li>
        <li class="page-item active" aria-current="page">
            <a class="page-link" href="#">1</a>
        </li>
        <li class="page-item">
            <a class="page-link" href="#">2</a>
        </li>
        <li class="page-item">
            <a class="page-link" href="#">3</a>
        </li>
        <li class="page-item">
            <a class="page-link" href="#">Next</a>
        </li>
    </ul>
</nav>
```

接下来，让我们根据上面的示例来简单介绍一下与该组件相关的样式类及其使用方法，具体如下：

- `pagination`：该样式类通常作用于`<nav>`标记下面的`<ul>`标记，效果是将该无序列表元素设置为分页组件，并赋予其该组件的基本样式；
- `page-item`：该样式类是`pagination`类的次级样式类，通常作用于分页组件中的每个`<li>`标记，效果是将这些列表项设置为该组件中跳转按钮的样式。在设置这些跳转按钮元素时，读者还需要注意以下事项：

        - 如果想让某个跳转按钮处于禁用状态，则需要在`page-item`类后面再添加一个`disabled`样式类；
        - 如果想将某个跳转按钮设置为默认激活状态，则需要在`page-item`类后面再添加一个`active`样式类；
    - `page-link`：该样式类是`page-item`类的次级样式类，通常被放置在被设置了`page-item`样式类的`<li>`标记的内部，作用是具体设置组件中各个分页所在的链接；

- **模态对话框组件**：如果读者想在页面中以弹出式对话框的方式来呈现某些内容（例如在本章项目中，笔者将会使用这种对话框的形式来呈现多媒体播放界面），则可以使用Bootstrap框架提供的模态对话框组件来进行辅助设计。下面是该组件的一个简单示例：

```
<button type="button" class="btn btn-primary"
    data-bs-toggle="modal" data-bs-target="#modalExample">
    显示模态弹框
</button>
<dialog class="modal fade" id="modalExample" tabindex="-1"
    aria-labelledby="modalExampleLabel" aria-hidden="true">
    <div class="modal-dialog">
        <div class="modal-content">
            <div class="modal-header">
                <h5 class="modal-title"
                    id="modalExampleLabel">
                    模态弹框标题
                </h5>
                <button type="button" class="btn-close"
                    data-bs-dismiss="modal"
                    aria-label="Close">
                </button>
            </div>
            <div class="modal-body">
                <p>这是一个模态弹框组件的简单示例。</p>
            </div>
            <div class="modal-footer">
                <button type="button"
                    class="btn btn-secondary"
                    data-bs-dismiss="modal">关闭</button>
                <button type="button"
                    class="btn btn-primary">
                    保存更改
                </button>
            </div>
        </div>
    </div>
</dialog>
```

接下来，笔者将根据上述示例来介绍一下该组件的使用方法以及相关的样式类，具体如下：

    - `modal`：该样式类通常作用于一个`<div>`或`<dialog>`标记，效果是将该标记定义的元素设置为一个模态对话框组件。在设置该组件时，读者需要注意以下事项：

        - 由于该组件是一个弹出式对话框，所以我们需要设置一个按钮元素来触发该对话框的弹出。在定义该按钮元素的`<button>`标记中，`data-bs-toggle`属性的值应设置为`modal`，`data-bs-target`属性的值则应为`#[id属性值]`（在这里，`[id属性值]`为模态对话框组件的`id`属性值）。
        - 如果我们想赋予该模态对话框淡入淡出的效果，就需要在`modal`样式类后面再添加一个`fade`样式类；
    - `modal-dialog`：该样式类是`modal`类的次级样式类，通常作用于被设置了`modal`类的`<div>`标记内第一级的`<div>`标记，效果是将该标记定义的元素设置为模态对话框组件的对话框部分。在设置这部分样式时，读者需要注意以下事项：
        - 如果读者想在对话框中使用滚动条元素，就需要在`modal-dialog`样式类后面再添加一个`modal-dialog-scrollable`样式类；
        - 如果读者想在对话框中使用居中布局，就需要在`modal-dialog`样式类后面再添加一个`modal-dialog-centered`样式类；
    - `modal-content`：该样式类是`modal-dialog`类的次级样式类，通常作用于被设置了`modal-dialog`类的`<div>`标记内第一级的`<div>`标记，效果是将该标记定义的元素设置为一个用于放置对话框内部元素的容器；
    - `modal-header`：该样式类是`modal-content`类的次级样式类，通常作用于被设置了`modal-content`类的`<div>`标记内第一级的`<div>`标记，效果是将该标记定义的元素设置为一个模态对话框组件的头部部分。
    - `modal-title`：该样式类是`modal-header`类的次级样式类，通常作用于被设置了`modal-header`类的`<div>`标记内的标题类标记，效果是将该标记定义的元素设置为一个模态对话框组件的标题文本。
    - `modal-body`：该样式类是`modal-content`类的次级样式类，通常作用于被设置了`modal-content`类的`<div>`标记内第一级的`<div>`标记，效果是将该标记定义的元素设置为一个模态对话框组件的身体部分。
    - `modal-footer`：该样式类是`modal-content`类的次级样式类，通常作用于被设置了`modal-content`类的`<div>`标记内第一级的`<div>`标记，效果是将该标记定义的元素设置为一个模态对话框组件的底部部分；

【工作实施和交付】

在完成了上述知识准备之后，读者现在就可以根据之前【任务书】中的要求来着手实施凌雪冰熊网站的“产品展示”页设计了，该项目的实施过程可以分为以下步骤来进行。

第1步：基于网页模板来创建目标页面

在这一步骤中，网页设计师的主要任务是基于刚刚创建的网页设计模板来创建网站的“产品展示”页，并完成该页面主体部分的内容布局。为此，读者需要执行以下操作。

1. 先使用命令行终端环境回到到凌雪冰熊网站项目的根目录下，并通过执行`cp template.htm show.htm`命令来创建该网站的“产品展示”页。

2. 使用VS Code编辑器打开刚刚创建的`show.htm`文件，找到页面的导航栏部分并将其当前页有“首页”改为“产品展示”，并将之前创建的网站首页、新闻活动页所在的URL添加到该导航栏相应的链接标记中，具体代码如下。

3. 最后回到之前的命令行终端环境中，并在项目的根目录中通过执行以下命令来完成本章项目的第一次版本控制操作。

```
git init
git add .
git commit -m "项目4：完成产品展示页的创建"
```

第2步：完成“产品展示”页的设计

在这一步骤中，网页设计师的主要任务是：先使用Bootstrap框架提供的选项卡组件和卡片组件对凌雪冰熊提供的视频资料建立播放列表界面，然后再为该播放列表中的每个视频资料提供播放入口，以便用于打开多媒体播放界面。为此，读者需要执行以下操作。

1. 继续使用VS Code编辑器回到`show.htm`文件中，并找到`<!-- 在此处填充网页的主体内容 -->`这一行注释所在的位置，并将其替换为以下代码。

```

```

2. 在保存上述代码，并使用网页浏览器打开`show.htm`文件之后，读者就可以查看当前网页设计的结果了，其外观在Google Chrome浏览器中的效果如图4-所示。

   图4-

   图4-：“产品展示”页的设计效果

3. 最后回到之前的命令行终端环境中，并在项目的根目录中通过执行以下命令来完成本章项目的第二次版本控制操作。

```
git add .
git commit -m "项目4：完成产品展示页的布局"
```

第3步：多媒体播放界面的设计

在这一步骤中，网页设计师的主要任务是使用HTML中的`<video>`标记及其子标记来创建用于播放多媒体资料的用户界面，并利用Bootstrap框架提供的样式类完成该用户界面的设计工作。为此，读者需要执行以下操作。

1. 继续使用VS Code编辑器回到`show.htm`文件中，并找到`<!-- 在此处填充视频播放器元素 -->`这一行注释所在的位置，并将其替换为以下代码。

```

```

2. 在保存上述代码之后，使用网页浏览器打开`show.htm`文件，并在打开的网页中单击任意一张卡片中带有“播放视频”字样的按钮，然后就可以查看当前网页设计的结果了，其外观在Google Chrome浏览器中的效果如图4-所示。

   图4-

图4-：多媒体播放界面的设计效果

3. 最后回到之前的命令行终端环境中，并在项目的根目录中通过执行以下命令来完成本章项目的第三次版本控制操作。

```
git add .
git commit -m "项目4：完成多媒体播放界面的设计"
```

【拓展知识】

【作业】

有一家名为“白熊前端”的程序员培训机构，刚刚完成了其官方网站的首页设计，现在希望你能根据首页制定的网页设计风格，继续为该网站设计一个“课程展示”页，用于以视频的方式向公众更好地展示该培训机构所能提供的服务，例如其明星级的教师团队、最近的热门课程等，以便吸引到更多的潜在客户和合作伙伴。

- **项目名**：白熊前端网站的“课程展示”页设计
- **委托方**：白熊前端的创始人：林宇一
- **项目资料**：
  - **代码资料**：白熊前端官方网站现有的设计源码；
  - **文献资料**：反映白熊前端这家培训机构近况的视频资料；
- **项目要求**：为白熊前端的官方网站设计“课程展示”页，该网页的设计应符合以下要求。
  - 该网页需要为视频资料提供用户体验良好的多媒体播放界面；
  - 该网页通用需要采用与网站首页一致的布局风格与配色方案；
- 时间要求：在3个工作日内完成；

【作业评价】

序号	评测内容	评分标准	分值	自评	互评	师评	综合得分
01	视频资料的呈现	网页中是否呈现了甲方提供的视频资料？	20				
02	多媒体界面的设计	网页中的视频播放功能是否实现了用户体验良好的操作界面？	20				
03	PC端浏览器呈现	网页在Chrome和Firefox这两款主流网页浏览器中的表现是否一致？	30				
04	移动端浏览器呈现	网页在iOS和Android的各型主流设备中的表现是否一致？	30				